

MACHINE LEARNING FUNDAMENTALS REPORT

Multinomial & Bernoulli Naïve Bayes — From Scratch

5-Document Toy Corpus · 15-Word Vocabulary · Manual DTM Construction · Transparent
Probability Walkthrough

Prince Maheta

Data Science | Machine Learning | NLP

GitHub: [princemaheta2627-glitch](#)

June 2025

Executive Summary

This report documents a foundational Naïve Bayes teaching project that applies both Multinomial and Bernoulli classifiers to a hand-crafted 5-document toy corpus with a 15-word vocabulary. Unlike the previous two SMS spam projects (Projects 15 and 16) that used 5,572 real messages and automated train-test splits, this project builds the Document-Term Matrix (DTM) step by step, displays it as a readable DataFrame, and makes predictions on a single test document — providing complete transparency into what CountVectorizer produces and how Naïve Bayes uses it.

The project classifies documents into two categories: Cinema (0) and Education (1). With just 5 training documents and 1 test document, every matrix cell, every vocabulary word, and every probability score can be manually verified. Both Multinomial NB (62.0% education probability) and Bernoulli NB (62.8% education probability) agree the test document is Education — and they are correct.

Project Highlights

Dimension	Value	Educational Purpose
Training Documents	5 sentences (3 Education, 2 Cinema)	Small enough to read every row manually

Dimension	Value	Educational Purpose
Test Document	1 sentence: 'very good educational institution'	Single prediction to trace through the algorithm
Vocabulary (no stop words)	15 unique words	Every word visible and traceable in the DTM
Vocabulary (with stop words)	19 unique words	Comparison shows which words stop word removal removes
DTM Shape (after stop words)	5 rows × 15 columns	Exact sparse matrix printed and displayed as DataFrame
MultinomialNB — P(Cinema)	37.97%	Computed and printed explicitly
MultinomialNB — P(Education)	62.03%	Model predicts Education ✓ (correct)
BernoulliNB — P(Cinema)	37.19%	Slightly different due to binary modelling
BernoulliNB — P(Education)	62.81%	Same prediction: Education ✓ (correct)
Key Skill Introduced	X_transformed.toarray() + pd.DataFrame display	Makes the abstract sparse matrix human-readable
Institution Featured	Future Vision Computers Institute	Appears in Document 1 of training data

1. Why This Project Matters

1.1 The Teaching Role of Toy Datasets

Projects 15 and 16 worked with 5,572 real SMS messages — the model operated at a scale where no human could manually verify the probability calculations. This project deliberately uses a 5-document corpus so every step can be traced by hand. This is the standard pedagogical approach in machine learning education: teach with small data first, scale up later.

The key questions this project answers transparently:

- What exactly does CountVectorizer produce? — the DTM DataFrame shows every word count for every document
- How does stop word removal change the vocabulary? — comparing before/after vocabularies makes this concrete
- What does the sparse matrix look like as numbers? — X_transformed.toarray() and print(X_transformed) show the raw structure
- What probability does the model assign? — predict_proba() output is displayed and interpreted
- Do Multinomial and Bernoulli agree? — both models are applied to the same test document and their probabilities compared

1.2 Position in the Portfolio

Project	Dataset	Scale	Focus
Project 15 — Multinomial NB	5,572 SMS messages	Large — real-world	Performance metrics (98.8% accuracy, ROC-AUC 0.992)
Project 16 — Bernoulli NB	5,572 SMS messages	Large — real-world	Model comparison (Multinomial vs Bernoulli trade-offs)
Project 17 — This project	5 toy documents	Tiny — educational	Algorithm transparency — see every cell, every word, every probability

Together, the three projects form a complete NB learning arc: understand the mechanism (Project 17) → apply at scale (Project 15) → compare algorithms (Project 16).

2. Dataset — The Toy Corpus

2.1 Training Data (example_train.csv)

The training set contains exactly 5 documents: 3 labelled Education and 2 labelled Cinema. The documents were created by Future Vision Computers Institute as a teaching example.

Doc #	Document Text	Class	Label Encoded
0	Future Vision Computer Institute is a great educational institution.	education	1
1	Educational greatness depends on ethics	education	1
2	A story of great ethics and educational greatness	education	1
3	Sholey is a great cinema	cinema	0
4	good movie depends on good story	cinema	0

2.2 Test Data (example_test.csv)

Doc #	Document Text	Class	Label Encoded	Correct?
0	very good educational institution	education	1	Yes ✓ (both models predict Education)

2.3 Label Encoding

```
train_docs['Class'] = train_docs.Class.map({'cinema':0, 'education':1})
# cinema → 0
```

```
# education → 1

# y_train = [1, 1, 1, 0, 0]
# 3 education docs (label=1), 2 cinema docs (label=0)
```

3. CountVectorizer — Vocabulary Construction

3.1 Without Stop Words — 19-Word Vocabulary

First, CountVectorizer is fitted without stop word removal. It learns all unique words from the 5 training documents:

```
vec = CountVectorizer() # no stop_words parameter
vec.fit(X_train)
vec.vocabulary_

# Output: {'future':6, 'vision':18, 'computer':2, 'institute':10,
#         'is':12, 'great':8, 'educational':4, 'institution':11,
#         'greatness':9, 'depends':3, 'on':15, 'ethics':5,
#         'story':17, 'of':14, 'and':0, 'sholey':16,
#         'cinema':1, 'good':7, 'movie':13}
```

Without Stop Words	Words Included	Count
Total vocabulary	All words including function words	19 words
Function/stop words included	is, on, of, and	4 words (21% of vocabulary)
Content words	future, vision, computer, institute, great, educational, institution, greatness, depends, ethics, story, sholey, cinema, good, movie	15 words

3.2 With Stop Words — 15-Word Vocabulary

Refitting with stop_words='english' removes the 4 function words (is, on, of, and):

```
vec = CountVectorizer(stop_words='english')
vec.fit(X_train)
vec.vocabulary_

# Output: {'future':5, 'vision':14, 'computer':1, 'institute':9,
#         'great':7, 'educational':3, 'institution':10,
#         'greatness':8, 'depends':2, 'ethics':4, 'story':13,
#         'sholey':12, 'cinema':0, 'good':6, 'movie':11}
```

The 15-word vocabulary is the active feature set. Every word has an assigned column index (0–14), which is how the sparse matrix is structured. The vocabulary is alphabetically ordered:

Index	Word	Appears in	Cinema / Education indicator
0	cinema	Doc 3 (Cinema)	Strong cinema signal
1	computer	Doc 0 (Education)	Education signal
2	depends	Docs 1, 4 (Education & Cinema)	Neutral — appears in both
3	educational	Docs 0, 1, 2 (Education)	Strong education signal
4	ethics	Docs 1, 2 (Education)	Strong education signal
5	future	Doc 0 (Education)	Education signal
6	good	Doc 4 (Cinema)	Cinema signal (appears twice)
7	great	Docs 0, 2, 3 (mixed)	Neutral — appears in both classes
8	greatness	Docs 1, 2 (Education)	Education signal
9	institute	Doc 0 (Education)	Education signal
10	institution	Docs 0 (Education)	Education signal
11	movie	Doc 4 (Cinema)	Strong cinema signal
12	sholey	Doc 3 (Cinema)	Strong cinema signal (film name)
13	story	Docs 2, 4 (mixed)	Appears in both classes
14	vision	Doc 0 (Education)	Education signal

4. Document-Term Matrix — The Full Picture

4.1 Sparse Matrix Output

```
X_transformed = vec.transform(X_train)
print(X_transformed)

# Output (sparse format — row, col): count:
# (0, 1)      1  <- Doc 0, 'computer', count=1
# (0, 3)      1  <- Doc 0, 'educational', count=1
# (0, 5)      1  <- Doc 0, 'future', count=1
# (0, 7)      1  <- Doc 0, 'great', count=1
# (0, 9)      1  <- Doc 0, 'institute', count=1
# (0,10)      1  <- Doc 0, 'institution', count=1
# (0,14)      1  <- Doc 0, 'vision', count=1
# (1, 2)      1  <- Doc 1, 'depends', count=1
# (1, 3)      1  <- Doc 1, 'educational', count=1
# (1, 4)      1  <- Doc 1, 'ethics', count=1
# (1, 8)      1  <- Doc 1, 'greatness', count=1
# (2, 3)      1  <- Doc 2, 'educational', count=1
# (2, 4)      1  <- Doc 2, 'ethics', count=1
# (2, 7)      1  <- Doc 2, 'great', count=1
```

```
# (2, 8)      1  <- Doc 2, 'greatness', count=1
# (2,13)     1  <- Doc 2, 'story', count=1
# (3, 0)      1  <- Doc 3, 'cinema', count=1
# (3, 7)      1  <- Doc 3, 'great', count=1
# (3,12)     1  <- Doc 3, 'sholey', count=1
# (4, 2)      1  <- Doc 4, 'depends', count=1
# (4, 6)      2  <- Doc 4, 'good', count=2  (appears twice!)
# (4,11)     1  <- Doc 4, 'movie', count=1
# (4,13)     1  <- Doc 4, 'story', count=1
```

Only 23 of 75 total cells (5 documents × 15 words) have non-zero values — a 69.3% sparsity rate. This is relatively low compared to real-world corpora where sparsity exceeds 99%.

4.2 DTM as a Dense DataFrame (Key Teaching Moment)

```
pd.DataFrame(X_transformed.toarray(), columns=vec.get_feature_names_out())

# Output:
# cinema  computer  depends  educational  ethics  future  good  great  \
# Doc 0:  0        1        0          1        0        1        0        1
# Doc 1:  0        0        1          1        1        0        0        0
# Doc 2:  0        0        0          1        1        0        0        1
# Doc 3:  1        0        0          0        0        0        0        1
# Doc 4:  0        0        1          0        0        0        2        0

# greatness  institute  institution  movie  sholey  story  vision
# Doc 0:  0        1        1          0        0        0        1
# Doc 1:  1        0        0          0        0        0        0
# Doc 2:  1        0        0          0        0        1        0
# Doc 3:  0        0        0          1        0        0        0
# Doc 4:  0        0        0          1        0        1        0
```

4.3 Reading the DTM Row by Row

The DTM tells the complete story of each document in numbers:

Document	Non-Zero Words	Word Counts	Class	What the Numbers Say
Doc 0	computer, educational, future, great, institute, institution, vision	All 1s	Education (1)	7 distinct content words — richest education document
Doc 1	depends, educational, ethics, greatness	All 1s	Education (1)	4 words; ethics and greatness are strong education signals

Document	Non-Zero Words	Word Counts	Class	What the Numbers Say
Doc 2	educational, ethics, great, greatness, story	All 1s	Education (1)	5 words; great and story appear in both classes
Doc 3	cinema, great, sholey	All 1s	Cinema (0)	Only 3 words; cinema and sholey are pure cinema signals
Doc 4	depends, good(x2), movie, story	good=2, rest=1	Cinema (0)	'good' appears twice — the only word count > 1 in the dataset

The 'good' word in Doc 4 having count=2 is the only non-binary value in this tiny dataset. This illustrates the exact difference between Multinomial NB (uses count=2, giving 'good' more weight) and Bernoulli NB (treats count=2 the same as count=1, as simply 'present').

5. Test Document — Transformation and Prediction

5.1 Test Document Transformation

```
X_test_transformed = vec.transform(X_test)
# Sparse matrix: <1x15> with 3 stored elements

X_test = X_test_transformed.toarray()
# Output: array([[0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0]])
#           cin com dep edu eth fut goo gre gns ins isn mov shl sto vis
#           0  0  0  1  0  0  1  0  0  0  1  0  0  0  0
```

The test document 'very good educational institution' maps to 3 non-zero cells:

Word in Test Doc	In Vocabulary?	Column Index	Count	Note
very	No (stop word removed)	N/A	0	'very' was removed by stop_words='english'
good	Yes	6	1	Cinema signal — appeared in Doc 4
educational	Yes	3	1	Strong education signal — appeared in Docs 0, 1, 2
institution	Yes	10	1	Education signal — appeared in Doc 0 only

The model must now decide: does 'educational' + 'institution' (education signals) outweigh 'good' (cinema signal)? Intuitively yes — both models agree, predicting Education with ~62% probability.

6. Multinomial NB — Prediction with Probability

6.1 Training and Predicting

```
from sklearn.naive_bayes import MultinomialNB
mnb = MultinomialNB()
mnb.fit(X_transformed, y_train)
proba = mnb.predict_proba(X_test)

print('probability of test document belonging to class CINEMA', proba[:,0])
print('probability of test document belonging to class EDUCATION', proba[:,1])

# Output:
# probability of test document belonging to class CINEMA [0.3797015]
# probability of test document belonging to class EDUCATION [0.6202985]
```

6.2 Manual Probability Walk-Through

With only 5 documents and 15 words, we can trace the Multinomial NB calculation by hand:

Prior Probabilities

Class	Training Count	Prior P(class)
Cinema (0)	2 of 5 documents	$P(\text{Cinema}) = 2/5 = 0.40$
Education (1)	3 of 5 documents	$P(\text{Education}) = 3/5 = 0.60$

Likelihood of Test Words Given Class

For Multinomial NB, we need $P(\text{word} \mid \text{class})$ for each word in the test document. With Laplace smoothing ($\alpha=1$):

```
# P(word | class) = (count of word in class + 1) / (total words in class + vocab_size)

# Education class word counts from DTM:
# 'educational': 1+1+1 = 3 times in Docs 0,1,2
# 'institution': 1 time in Doc 0
# 'good': 0 times in any Education doc
# Total words in Education docs (approx): 7+4+5 = 16 tokens
# Vocab size = 15

# P(educational | Education) ≈ (3+1)/(16+15) = 4/31 ≈ 0.129 (strong signal)
# P(institution | Education) ≈ (1+1)/(16+15) = 2/31 ≈ 0.065 (present signal)
# P(good | Education) ≈ (0+1)/(16+15) = 1/31 ≈ 0.032 (absent but smoothed)

# P(educational | Cinema) ≈ (0+1)/(7+15) = 1/22 ≈ 0.045 (absent signal)
```



```
# P(institution | Cinema)  ≈ (0+1)/(7+15)  = 1/22 ≈ 0.045  (absent signal)
# P(good | Cinema)        ≈ (2+1)/(7+15)  = 3/22 ≈ 0.136  (strong cinema signal)
```

Combined score comparison (log space to avoid underflow):

	Log P(Cinema test)	Log P(Education test)	Conclusion
Prior	$\log(0.40) = -0.916$	$\log(0.60) = -0.511$	Education starts ahead
P(educational class)	$\log(0.045) = -3.10$	$\log(0.129) = -2.05$	Education gains +1.05 log units
P(institution class)	$\log(0.045) = -3.10$	$\log(0.065) = -2.73$	Education gains +0.37 log units
P(good class)	$\log(0.136) = -2.00$	$\log(0.032) = -3.44$	Cinema gains +1.44 log units
Total (approx)	~-9.02	~-8.73	Education wins → predicts Education ✓

The two education words ('educational' and 'institution') together outweigh the single cinema word ('good'). The final probabilities of 37.97% Cinema vs 62.03% Education reflect this balance.

7. Bernoulli NB — Prediction with Comparison

7.1 Training and Predicting

```
from sklearn.naive_bayes import BernoulliNB
bnb = BernoulliNB()
bnb.fit(X_transformed, y_train)
prob_bnb = bnb.predict_proba(X_test)
print(prob_bnb)

# Output: array([[0.37190622, 0.62809378]])
# Cinema: 37.19% | Education: 62.81%
```

7.2 Why the Probabilities Differ from Multinomial

Both models predict Education, but their exact probabilities differ:

Model	P(Cinema)	P(Education)	Difference from Multinomial
Multinomial NB	37.97%	62.03%	Baseline
Bernoulli NB	37.19%	62.81%	Education probability 0.78pp higher

The Bernoulli model gives Education a slightly higher probability (62.81% vs 62.03%). Why? Bernoulli NB also models absent words:

- The 12 vocabulary words absent from the test document also contribute to the calculation
- Education-marker words that are ABSENT from the test document (future, computer, ethics, vision, greatness...) pull the cinema score down more than the education score, because these absent words are strongly education-associated
- This absent-word signal is unique to Bernoulli NB — Multinomial NB ignores absent words entirely
- The 'good' word appearing once vs twice (count=2 in training) makes no difference to Bernoulli NB — it binarises counts. This slightly changes the cinema probability calculation vs Multinomial

8. Complete Pipeline Summary

8.1 Full Code Walkthrough

```
import numpy as np, pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB

# 1. Load data
train_docs = pd.read_csv('example_train.csv')
test_docs = pd.read_csv('example_test.csv')

# 2. Encode labels
train_docs['Class'] = train_docs.Class.map({'cinema':0, 'education':1})
test_docs['Class'] = test_docs.Class.map({'cinema':0, 'education':1})

# 3. Split features and target
X_train = train_docs.values[:,0] # 5 document strings
y_train = train_docs.values[:,1].astype('int') # [1,1,1,0,0]
X_test = test_docs.values[:,0] # 1 document string
y_test = test_docs.values[:,1] # [1]

# 4. CountVectorizer (with stop words)
vec = CountVectorizer(stop_words='english')
vec.fit(X_train) # Learn 15-word vocabulary
X_transformed = vec.transform(X_train) # 5x15 sparse matrix
X_test_transformed = vec.transform(X_test) # 1x15 sparse matrix

# 5. Inspect DTM
print(vec.get_feature_names_out()) # ['cinema', 'computer', ..., 'vision']
print(len(vec.get_feature_names_out())) # 15
pd.DataFrame(X_transformed.toarray(), columns=vec.get_feature_names_out())
```

```
# 6. Train and predict - Multinomial NB
mnb = MultinomialNB()
mnb.fit(X_transformed, y_train)
proba_mnb = mnb.predict_proba(X_test_transformed)
# [[0.37970, 0.62030]] → Education

# 7. Train and predict - Bernoulli NB
bnb = BernoulliNB()
bnb.fit(X_transformed, y_train)
proba_bnb = bnb.predict_proba(X_test_transformed)
# [[0.37191, 0.62809]] → Education
```

9. Key Learnings & Concepts

9.1 New Concepts Introduced

- `train_array = train_docs.values` — converting a DataFrame to a numpy array for row/column indexing
- `X_train = train_array[:,0]` — slice all rows, column 0 (document text)
- `y_train = y_train.astype('int')` — explicit integer cast since `.values` from mixed DataFrame returns object dtype
- `X_transformed.toarray()` — converts sparse CSR matrix to dense numpy array for display and direct inspection
- `pd.DataFrame(X_transformed.toarray(), columns=vec.get_feature_names_out())` — the single most educational line in the project: makes the abstract sparse matrix human-readable with word labels
- Direct comparison of `predict_proba()` output from `MultinomialNB` and `BernoulliNB` on the same test document
- Manual verification: probability scores can be cross-checked by hand on this tiny dataset

9.2 Vocabulary Comparison — Stop Words in Action

Without Stop Words	With Stop Words	Removed (stop words)
19 words	15 words	is, on, of, and
includes: is, on, of, and	excludes these 4	These 4 add no classification signal
Example: 'Sholey is a great cinema'	Keeps: sholey, great, cinema	Removes: is, a

After stop word removal, the vocabulary contains only content words — words that carry meaning and discriminate between classes. Function words like 'is', 'on', 'of', 'and' appear equally in both Cinema and Education documents, so removing them makes the model more focused and accurate.

9.3 The DTM — The Heart of NLP ML

The Document-Term Matrix (DTM) is the central data structure in text machine learning. Every subsequent NLP model (TF-IDF, word embeddings, BERT) is essentially a more sophisticated version of the DTM — converting text into numbers in different ways. Understanding the DTM at this tiny scale (5 documents $\times$ 15 words) makes the concept concrete before scaling up to 5,572 messages $\times$ 7,204 words.

9.4 Both Models Agree — But When Do They Disagree?

On this test document, both models predict Education with ~62% probability. They agree because the test document has strong education markers (educational, institution) that outweigh the cinema marker (good). Models disagree more on borderline cases:

- A document with equal numbers of cinema and education words — the count-weighted Multinomial model might tip differently from the binary Bernoulli model
- A short document with one word that appears multiple times — Multinomial amplifies repeated words; Bernoulli ignores the repetition
- A document where many education-exclusive words are absent — Bernoulli notices the absence and adjusts; Multinomial ignores it

10. Conclusion

10.1 Summary

This project delivers the clearest possible picture of how Naïve Bayes text classification works internally — from raw text to vocabulary to DTM to probability scores. Every cell of the 5 $\times$ 15 Document-Term Matrix is visible and interpretable. The stop word comparison shows precisely which words are removed and why. The side-by-side Multinomial vs Bernoulli prediction (37.97/62.03 vs 37.19/62.81) confirms that both models agree on the classification while producing slightly different probability estimates due to their different treatment of word counts and absent words.

Both models correctly predict the test document 'very good educational institution' as Education — a satisfying result that validates the models' learning from just 5 training examples.

10.2 Portfolio Significance

- Closes the NLP theory loop: Projects 15 and 16 showed that the models work at scale; this project shows HOW they work internally
- Introduces `.toarray()` + `pd.DataFrame` visualisation — the standard way to inspect what a vectorizer has produced
- Demonstrates the stop word before/after comparison explicitly — a key NLP preprocessing concept
- Manual probability verification establishes statistical intuition: prior probabilities, word likelihoods, and Laplace smoothing all make intuitive sense on this small example

- Reinforces that Naïve Bayes needs very little data to learn: even with just 5 training documents, it correctly classifies the test document

11. Project Information

Project Title	Multinomial & Bernoulli Naïve Bayes — From Scratch with Toy Dataset
Author	Prince Maheta
GitHub Repository	github.com/princemaheta2627-glitch/Multinomial_And_Bernoulli_Naive_Bayes
Notebook	multinomial_and_bernouli_Naive_bayes_Answer.ipynb — 28 cells
Training Dataset	example_train.csv — 5 documents: 3 Education, 2 Cinema
Test Dataset	example_test.csv — 1 document: 'very good educational institution' (Education)
Vocabulary Size	15 words (after English stop word removal); 19 words (without)
DTM Shape	5 rows × 15 columns; 23 non-zero cells; 69.3% sparsity
MultinomialNB Result	P(Cinema)=37.97%, P(Education)=62.03% → Predicts Education ✓
BernoulliNB Result	P(Cinema)=37.19%, P(Education)=62.81% → Predicts Education ✓
Key Techniques	CountVectorizer comparison (with/without stop words), toarray() + DataFrame display, manual DTM reading
Project Type	NLP Text Classification — Foundational Understanding (Toy Dataset)
Institution	Future Vision Computers Institute (dataset creator)
Date Completed	June 2025

Report Date	June 26, 2026
--------------------	---------------

This report was prepared as part of a data science & machine learning portfolio.

Prince Maheta | Data Science Portfolio | 2025